

Clase 4

Memoria Caché



¿Por qué necesitamos una Caché?

El Problema

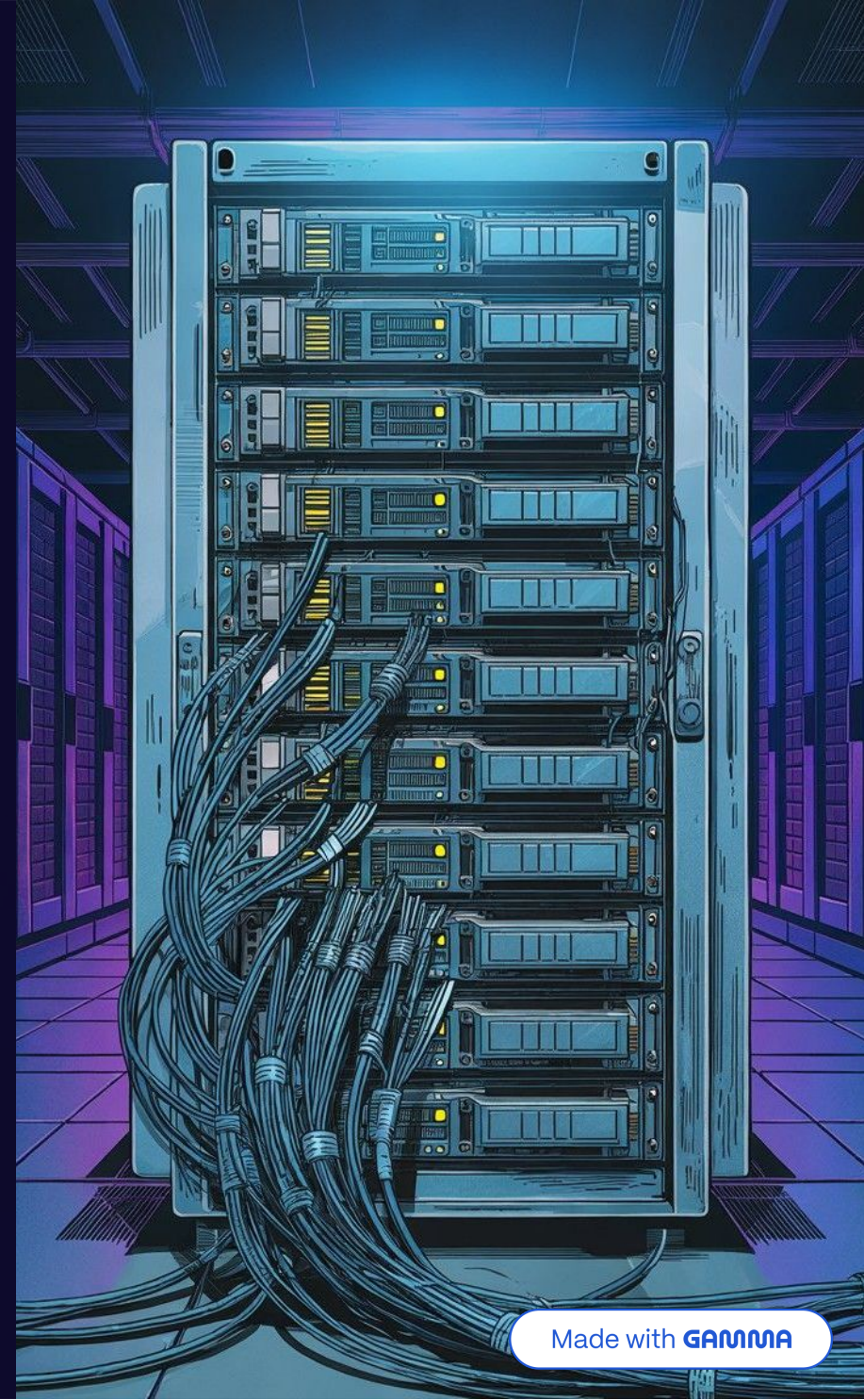
Las BD en disco (MongoDB) son excelentes para persistir datos, pero **lentas para lecturas masivas** comparadas con la RAM.

Caso de Uso Real

Cientos de usuarios consultando alojamientos simultáneamente — picos de tráfico en temporada alta: Salinas Gran Hotel, cabañas en Mina Clavero.

La Solución

Hacer **1,000 queries idénticas** a la BD desperdicia recursos. La caché guarda la primera consulta y sirve las 999 restantes desde memoria en **microsegundos**.



Configuración y Gestión de Caché

Para un funcionamiento óptimo, la caché requiere una configuración y gestión cuidadosas, abarcando aspectos clave como el tiempo de vida de los datos, las políticas de evicción y los límites de capacidad.

Tiempo de Vida (TTL - Time to Live)

Define cuánto tiempo un dato se considera válido. Se puede setear un máximo absoluto o setear tiempos de vida que se resetean cada vez que el dato es consultado.

Políticas de reemplazo

Reglas para liberar memoria cuando la caché alcanza su límite. Se pueden usar distintas estrategias: eliminar los mas antiguos, los menos consultados, los que llevan mas tiempo sin ser consultados.

Tamaño máximo (Max Memory)

Restricción física de RAM asignada al servicio de caché para evitar que consuma todos los recursos del servidor.

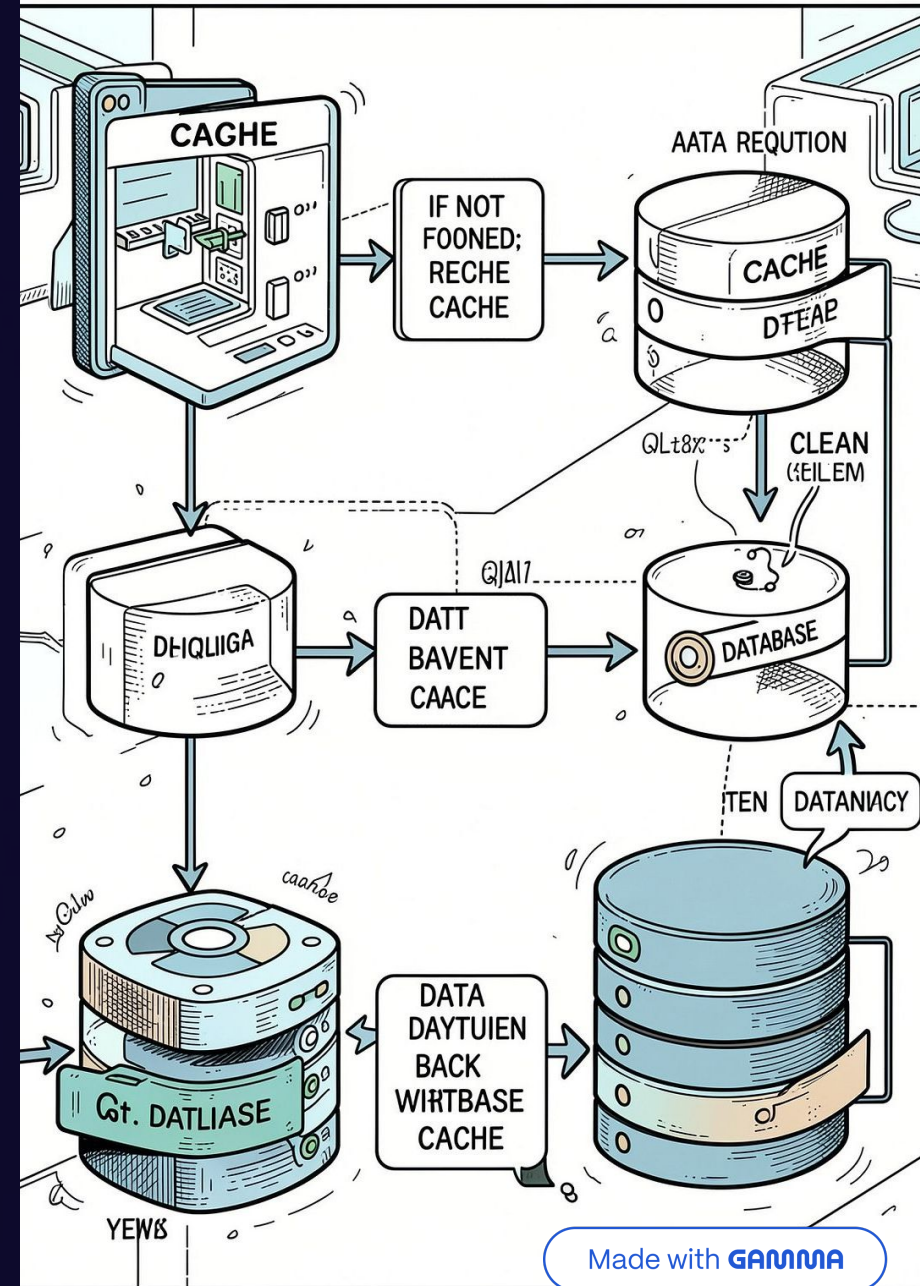
Claves

Como se identifican los datos. Funciona como diccionario. Claves suelen incluir tipo de dato además del id. Por ejemplo: PRODUCTO:12

Cache-Aside

- La aplicación consulta primero la caché.
- Si no encuentra el dato, consulta la BD.
- Luego guarda el resultado en caché.

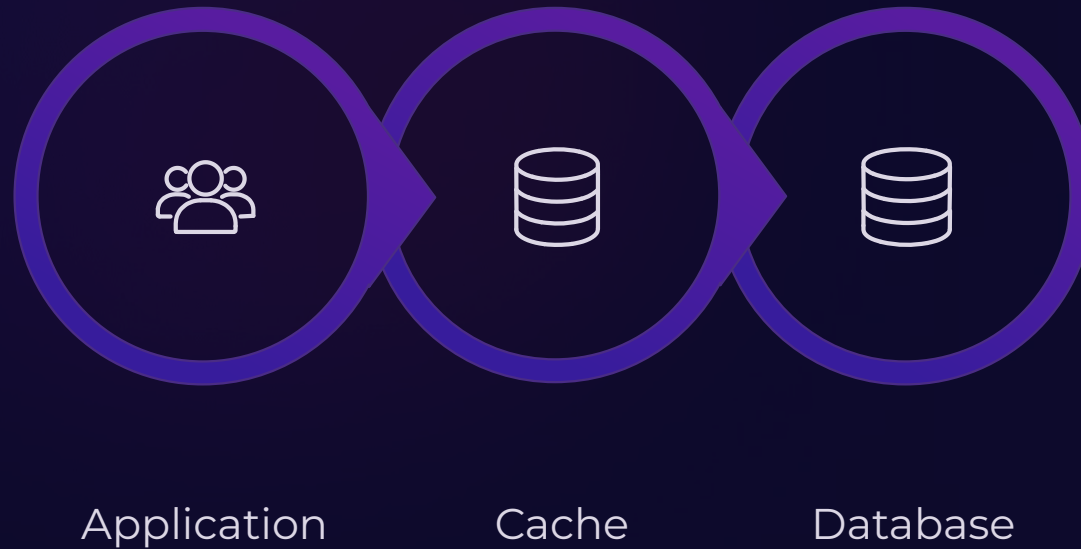
Flujo:



Read-Through

La aplicación consulta siempre la caché. Si el dato no está, la caché lo obtiene de la BD. La aplicación no gestiona directamente la consulta a la BD.

Flujo:

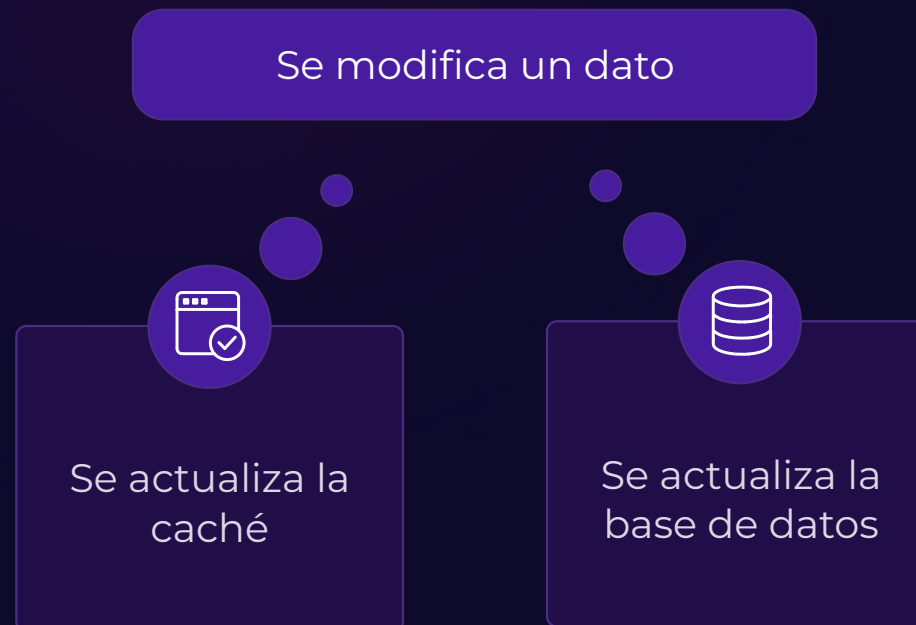




Write-Through

Al modificar un dato, se actualizan caché y BD. La caché queda actualizada inmediatamente. Prioriza la consistencia.

Flujo:





Write-Behind (Write-Back)

- Primero se actualiza la caché.
- La BD se actualiza posteriormente.
- Permite escrituras más rápidas.
- Existe riesgo si la caché falla antes de persistir el dato.

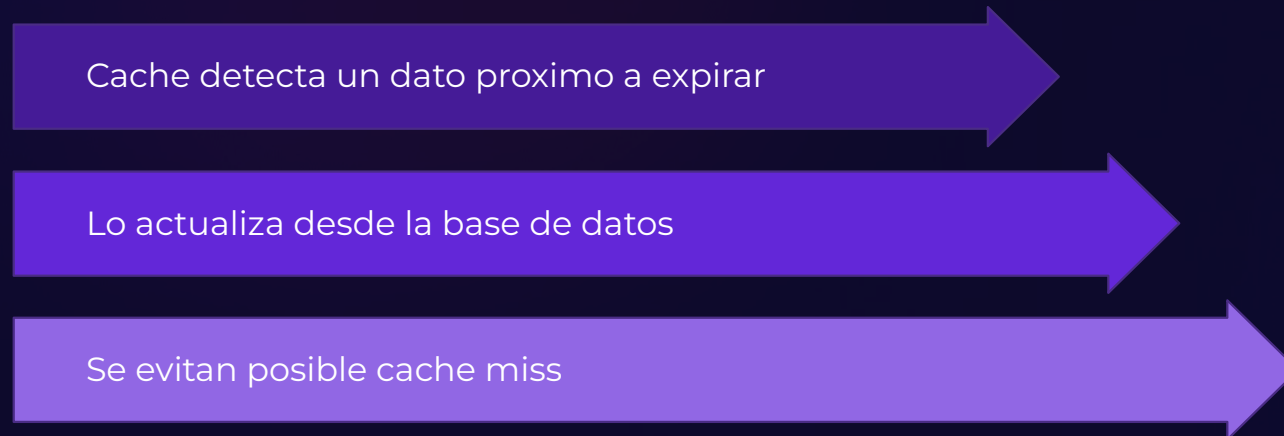
Flujo:



Refresh-Ahead

- La caché se actualiza antes de que el dato expire.
- Evita cache misses.
- Mantiene las lecturas rápidas.

Flujo:



¿Dónde guardamos la Caché?

Arquitectura: Local vs Distribuida

⚡ Caché Local (ccache)

Vive en la RAM de la propia API Go.

- Rapidísima, cero latencia de red
- Fácil de integrar
- ⚠ Se desincroniza con múltiples servidores

🌐 Caché Distribuida (Memcached)

Servicio externo independiente.

- Consistencia total entre instancias
- ⚠ Latencia de red y dependencia externa

Resiliencia y Degradación Elegante

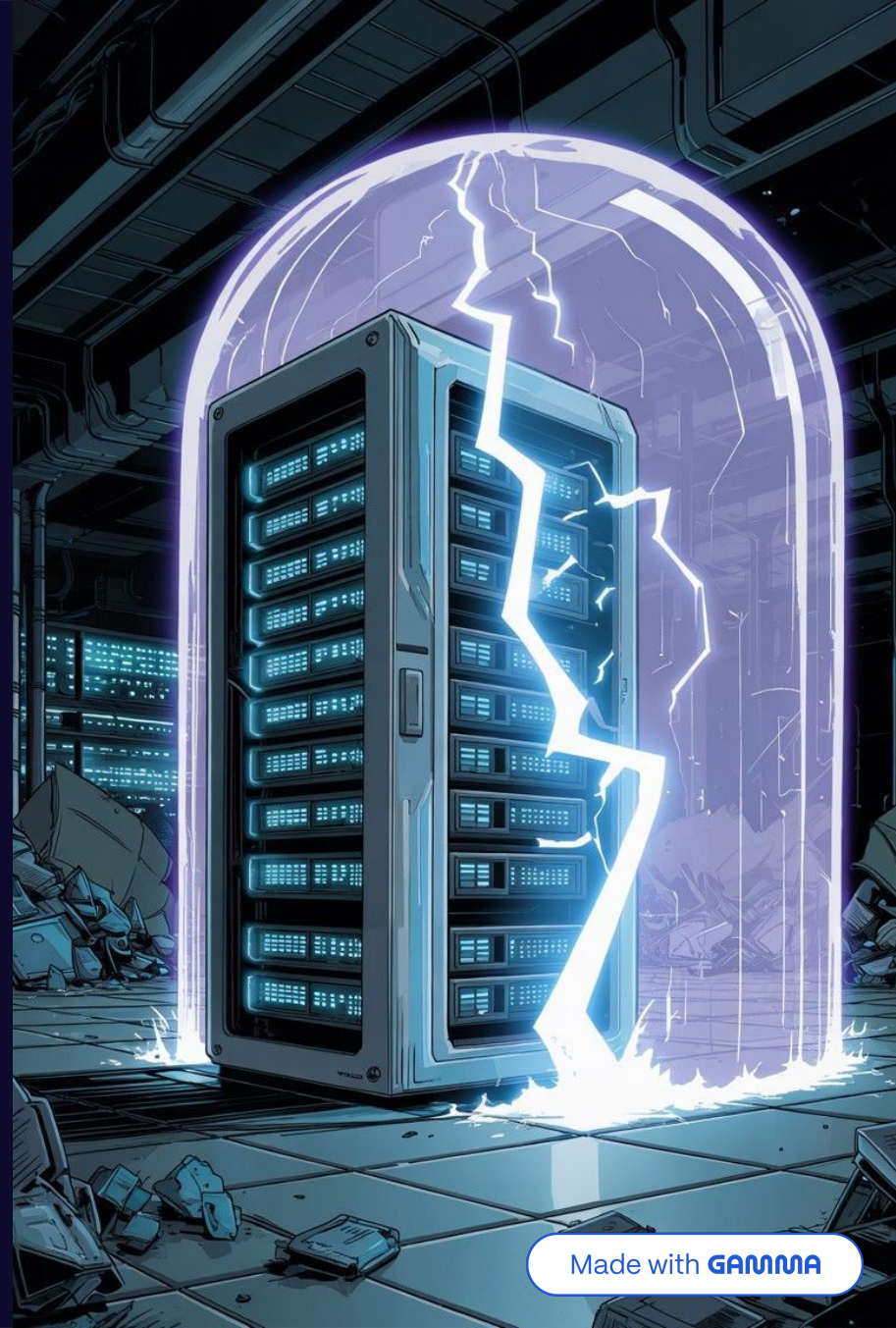
⚠ **Principio clave:** La caché **NO es la fuente de verdad**. Si la cache falla o se cae, la aplicación **no debe fallar**.

✗ Error Fatal

Devolver **HTTP 500** al cliente por un fallo de caché. Inaceptable en producción.

✓ Degradación Elegante

Capturar el error en Go, **loguearlo** e ignorar la caché temporalmente. Se busca directamente en la base de datos: más lento, pero el sistema **sigue vivo**.



Ejercicio 1: Medir el Problema

Consigna

- Agrega latencia artificial a la base de datos
- Simula una consulta lenta con `time.Sleep(500 * time.Millisecond)`
- Ejecuta varias veces la consulta del mismo producto
- Observa y mide el tiempo de respuesta en cada llamada

Código Clave

```
time.Sleep(500 * time.Millisecond)
```

Objetivo

- Establecer una línea base de latencia
- Comparar posteriormente con caché
- Entender el costo real de consultas a BD



Ejercicio 2: Implementar Local Cache

Consigna

- Implementa Cache-aside con `cache` (github.com/karlseguin/ccache)
- Patrón: Cache hit → devolver dato | Cache miss → consultar BD → guardar en caché
- Prueba el endpoint nuevamente
- Compara tiempos con la versión sin caché

Código Clave

```
value := cache.Get(key)
if value != nil {
    // cache hit
}
// cache miss → consultar BD
cache.Set(key, value, expiration)
```

Objetivo

- Observar diferencia entre cache hit y cache miss
- Medir mejora de latencia en memoria local
- Entender el patrón Cache-aside



Ejercicio 3: Implementar Memcached

Consigna

- Reemplaza la caché local por Memcached (github.com/bradfitz/gomemcache)
- Implementa nuevamente Cache-aside con caché externa
- Compara latencias: sin caché vs local cache vs Memcached

Setup: Docker

- Ejecuta Memcached en un contenedor Docker:

```
docker run --name memcached -p 11211:11211 -d memcached
```

- Memcached estará disponible en localhost:11211

Cliente Go

```
client := memcache.New("localhost:11211")
```

Código Clave

```
item, err := client.Get(key)
// cache miss → consultar BD
client.Set(&memcache.Item{
    Key: key,
    Value: value,
})
```

Flujo

- GET → Memcached
- Miss → Consulta BD
- Guardar en Memcached

Objetivo

- Observar diferencias entre caché local y externa
- Entender latencia de red vs latencia de proceso
- Medir impacto en rendimiento

Ejercicio 4: Tolerancia a Fallos

Consigna

- Simula la caída de Memcached: `docker stop memcached`
- La API debe seguir funcionando sin errores 500
- Implementa fallback: si Memcached falla, consulta BD directamente

Código Clave

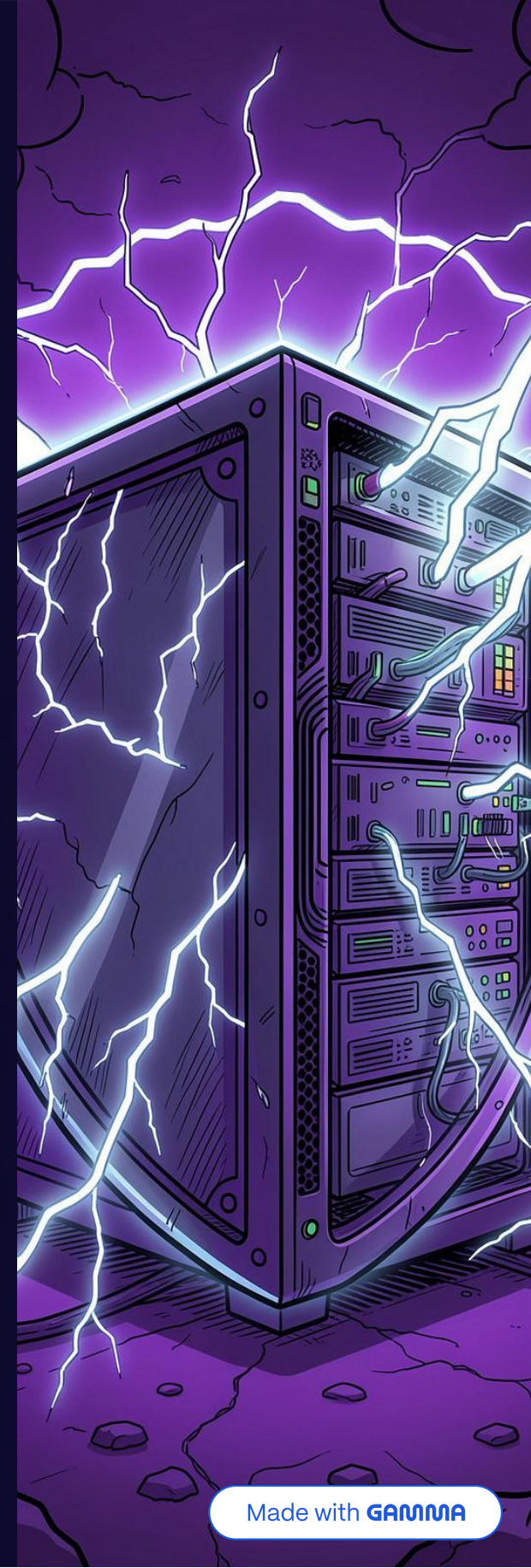
```
item, err := client.Get(key)
if err != nil {
    // fallback → consultar BD
}
```

Pregunta Clave

- ¿Qué ocurre si falla la escritura en Memcached?
- ¿Debería retornar error o ignorarlo?
- ¿Cómo afecta esto a la respuesta de la API?

Objetivo

- Entender que caché mejora rendimiento, no es crítica
- Implementar degradación elegante
- Construir sistemas resilientes



Recap & Q&A

Puntos de Cierre:

- Importancia de la separación de responsabilidades usando Interfaces en Go
- Optimización drástica de recursos y latencia
- Construcción de software backend verdaderamente tolerante a fallos

